

venn-diagram-lab — Python Package

User Guide

Zoltán Dul, Márton Ölbei, N. Shaun B. Thomas, Azeddine Si Ammour, Attila Csikász-Nagy

v2.4.1 — 2026-06-09

Contents

Overview	3
Installation	4
Quick start	4
From the CLI	4
From Python	4
Core concepts	5
Bundled sample datasets	6
Python API reference	6
I/O loaders	6
load_csv(path, *, binary=True, delimiter=None, prefix_cols=1) -> Dataset	6
load_tsv(path, *, binary=True, prefix_cols=1) -> Dataset	6
load_gmt(path) -> Dataset	6
load_gmx(path) -> Dataset	7
load_sample(name) -> Dataset	7
Catalog	7
list_samples() -> list[str]	7
list_models() -> list[ModelInfo]	7
Analysis	7
analyze(dataset, model="auto") -> RegionResult	7
Region accessors and Boolean DSL	7
intersection_items(result, sets)	8
exclusive_items(result, sets)	8
union_items(result, sets)	8
parse_region_expression(expr, n_sets)	8

CLI surface	9
Statistics (lower-level functions)	9
Item share distribution & cluster ordering	10
item_share_distribution(matrix) -> dict[int, int]	10
cluster_set_order(D, *, method="average") -> ClusterOrder	10
SVG renderers	10
render_venn_svg(result, *, model=None, set_names=None, colors=None, title=None, show_names=True, show_counts=True, show_items=False, item_options=None, highlight=None)	-> SvgImage 11
render_share_distribution_svg(dataset)	-> SvgImage 11
render_cluster_heatmap_svg(result, *, linkage="average", show_row_dendrogram=True, show_col_dendrogram=True, dendrogram_fraction=0.12)	-> SvgImage 11
render_enrichment_bar_svg(result, *, metric="neglog10fdr", width=560, height=240)	-> SvgImage 12
render_enrichment_lollipop_svg(result, *, metric="neglog10fdr", width=560, height=240)	-> SvgImage 12
generate_proportional_svg(result, *, width=600, height=600)	-> str 12
Matplotlib integration	12
render_venn_mpl(result, ax=None, model=None, set_names=None, colors=None, title=None, show_names=True, show_counts=True, alpha=0.45)	12
render_share_distribution_mpl(dataset, ax=None, color_low="#ffe4b5", color_high="#7e14ff")	13
render_cluster_heatmap_mpl(result, ax=None, linkage="average", show_row_dendrogram=True, show_col_dendrogram=True, cmap="Blues")	13
CLI --backend flag	13
SvgImage Jupyter inline display	13
ggplot/matplotlib renderers	14
render_upset(result, *, max_columns=20, sort_by="size", threshold=None, color_mode="depth", colors=None)	-> MplImage 14
render_network(result, *, edge_metric="intersection", seed=42, significance_threshold=0.05, node_color_map=None)	-> MplImage 14
Reports	14
render_pdf_report(result, path, *, title=None, include_network=True, include_about=True, cluster_heatmap=False)	-> None 14
to_excel_workbook(result, path)	-> None 15
Proportional layout	15
solve_2set(a_only, b_only, ab)	-> ProportionalLayout 15
solve_3set(regions)	-> ProportionalLayout 15

<code>circle_intersection_area(r1, r2, d) -> float</code>	15
Data classes (<code>@dataclass</code> and <code>TypedDict</code>)	15
Exceptions	17
CLI reference (vdl)	17
Top-level shortcuts	17
<code>vdl data</code>	18
<code>vdl export</code>	18
<code>vdl model</code>	19
<code>vdl render</code>	19
<code>vdl report</code>	20
<code>vdl workflow</code>	20
YAML config schema (<code>workflow run-from</code>)	21
Common workflows	21
End-to-end PDF report from a TSV	21
Full bundle (PDF + SVGs + TSVs + xlsx + README) for a down- stream user	22
CI parity test: compare two TSV exports byte-by-byte	22
Significance enrichment in a notebook	22
Custom 4-letter colour mapping in the Venn	22
Output formats	22
Cross-package parity	23
Troubleshooting	24
References	25
Citation	26

Overview

`venn-diagram-lab` is the headless Python companion to the *Venn Diagram Lab* web tool (<https://venndiagramlab.org/>). It exposes the same set-analysis math, the same 44 SVG models, the same PDF report layout, and a byte-equivalent TSV/Excel export contract — but without a browser. Two surfaces:

- **Python library** (`venn_diagram_lab`): pure functions and `dataclass`/typed-dict records you call from notebooks, scripts, pipelines.
- **Typer CLI** (`vdl`): every library surface is also reachable as a shell command, with the same defaults and the same output bytes.

Sister projects:

Surface	URL / Install	Notes
Web tool	https://venndiagramlab.org/	Browser SVG editor + interactive analysis
R companion	<code>install.packages("vennDiagramlab")</code>	flavoured port; same TSV bytes
Python (this)	<code>pip install venn-diagram-lab</code>	Headless, scriptable

All three are kept in lock-step at the same major version.

Installation

```
pip install venn-diagram-lab
```

Optional extras:

```
pip install "venn-diagram-lab[dev]" # pytest, ruff, mypy, ipykernel, nbconvert, build, twine
```

Verify:

```
vd1 version # prints "2.4.1"
python -c "import venn_diagram_lab as v; print(v.__version__)"
```

Hard requirements (pulled in automatically): pandas>=2.0, numpy>=1.24, scipy>=1.11, shapely>=2.0, matplotlib>=3.8, networkx>=3.0, lxml>=5.0, cairosvg>=2.7, typer>=0.12, rich>=13.0, pyyaml>=6.0, openpyxl>=3.1.

Quick start

From the CLI

```
# 1. Pick a bundled demo dataset and write a full PDF report next to it.
vd1 report pdf --sample --out /tmp/demo.pdf
```

```
# 2. Or do everything: PDF + 4 SVGs + 3 TSVs + xlsx + README in one ZIP.
vd1 report zip --sample --out /tmp/demo.zip
```

```
# 3. Or render a single artefact and pipe the SVG straight to stdout.
vd1 render venn --sample --out -
```

From Python

```
from venn_diagram_lab import load_sample, analyze, render_pdf_report

dataset = load_sample("dataset_real_cancer_drivers_4")
result = analyze(dataset) # 4 sets -> 15 regions + stats
```

```
render_pdf_report(result, "out/report.pdf") # multi-page PDF
print(result.statistics.jaccard)           # 4x4 DataFrame
```

Core concepts

Concept	Type	Holds
Dataset	<code>venn_diagram_lab.Dataset</code>	Set names, item lists, item order, universe size, source metadata
RegionResult	<code>venn_diagram_lab.RegionResult</code>	Dataset + per-region item lists + per-region masks + statistics
RegionData	<code>venn_diagram_lab.RegionData</code>	One region: bitmask, label ("AB"), exclusive items, inclusive items
StatisticsResult	<code>venn_diagram_lab.StatisticsResult</code>	Statistics: DataFrames for jac-card/dice/overlap/fold_enrichment + long-form hypergeometric table
ModelInfo	<code>venn_diagram_lab.ModelInfo</code>	One bundled SVG model's metadata (<code>name</code> , <code>n_sets</code> , <code>family</code>)
SvgImage	<code>venn_diagram_lab.SvgImage</code>	<code>svg: str</code> , <code>width: int</code> , <code>height: int</code> returned by SVG renderers
MplImage	<code>venn_diagram_lab.MplImage</code>	<code>fig: matplotlib.figure.Figure</code> returned by matplotlib renderers

A **region** is one of the $2^n - 1$ non-empty subsets of an n -set diagram. Each region has:

- a **bitmask** — 1 for set A only, 2 for set B only, 3 for A & B, 7 for A & B & C, etc.
- a **label** — the concatenation of member letters, e.g. "A", "AB", "ABC"
- **exclusive items** — items in this region and *only* this region
- **inclusive items** — items in this region OR any of its supersets

Bundled sample datasets

Five curated datasets ship with the package (3 real biological, 2 mock). All are 4-set TSVs (binary item-by-set matrices), suitable for every public function:

```
vd1 data samples
# dataset_real_cancer_drivers_4
# dataset_real_kegg_pathway_genes_4
# dataset_real_mirna_targets_4
# dataset_mock_overlap_4
# dataset_mock_disjoint_4
```

Or from Python:

```
from venn_diagram_lab import list_samples, load_sample
print(list_samples()) # ['dataset_real_...', ...]
ds = load_sample("dataset_real_cancer_drivers_4") # -> Dataset
```

Python API reference

I/O loaders

All five loaders return a `Dataset` ready for `analyze()`.

```
from venn_diagram_lab import load_csv, load_tsv, load_gmt, load_gmx, load_sample
```

load_csv(path, *, binary=True, delimiter=None, prefix_cols=1) -> Dataset

Load a comma-separated input. With `binary=True` (default) the file is a wide-form item×set matrix with 0/1 cells; with `binary=False` items are listed under set-named columns (one item per cell). `delimiter=None` auto-detects between `,` and `;`. `prefix_cols=1` means the first column is the item ID — set higher for files with multiple identifier columns.

```
ds = load_csv("genes.csv", binary=True, prefix_cols=1)
```

load_tsv(path, *, binary=True, prefix_cols=1) -> Dataset

Same contract as `load_csv` but tab-separated (the canonical webtool format).

```
ds = load_tsv("genes.tsv", binary=True)
```

load_gmt(path) -> Dataset

Load a Broad GMT file (tab-separated, one set per line: `set_name<TAB>description<TAB>item1<TAB>item2...`). Common in MSigDB-style gene-set distributions.

load_gmx(path) -> Dataset

Load a Broad GMX file (column-oriented sets, row 1 = set names, row 2 = descriptions, remaining rows = items).

load_sample(name) -> Dataset

Load one of the bundled curated demo datasets by stem name (no path, no extension).

```
from venn_diagram_lab import list_samples
for name in list_samples():
    ds = load_sample(name)
    print(name, ds.set_names, len(ds.item_order))
```

Catalog

list_samples() -> list[str]

Return the names of bundled demo datasets in deterministic order.

list_models() -> list[ModelInfo]

Return metadata for all 44 bundled Venn-diagram models. Each `ModelInfo` exposes `name` (filename stem), `n_sets` (2-9), and `family` (Venn / Edwards / Anderson / ...).

```
from venn_diagram_lab import list_models
for m in list_models():
    print(f"{m.name:30s} {m.n_sets} sets ({m.family})")
```

Analysis

analyze(dataset, model="auto") -> RegionResult

Compute all $2^n - 1$ regions plus pairwise statistics. With `model="auto"` the canonical Venn model for the set count is picked (`venn-2-set`, `venn-3-set`, `venn-4-set`, etc.). Pass an explicit model name (e.g. `"venn-5a-set-edwards"`) to render with a different layout — the math is layout-independent.

```
result = analyze(dataset)
result.statistics.jaccard           # 4x4 DataFrame
result.statistics.hypergeometric    # long-form: set_a, set_b, intersection,
                                     # expected, p_value, p_adjusted, significant
result.regions[7].exclusive_items  # A & B & C exclusive items
```

Region accessors and Boolean DSL

Four composable helpers for selecting items and regions inside an analysed diagram. All four are pure Python, no new dependencies, and chain naturally with

the existing renderers.

intersection_items(result, sets)

Items appearing in every set listed in **sets**, regardless of whether they also appear in other (unlisted) sets. **sets** is a sequence of set letters ("A", "B", ...) or display names (the values of **result.dataset.set_names**); the two forms may be mixed.

```
res = analyze(load_sample("dataset_real_cancer_drivers_4"))
intersection_items(res, ["Vogelstein", "COSMIC_CGC", "OncoKB"])
```

exclusive_items(result, sets)

Items in EXACTLY this combination — in every set in **sets** AND in none of the other sets in the dataset.

```
exclusive_items(res, ["Vogelstein", "COSMIC_CGC"])
```

union_items(result, sets)

Items appearing in any of the named sets (deduplicated). Returns items in **dataset.item_order** for deterministic byte-equivalent output.

parse_region_expression(expr, n_sets)

Small Boolean expression parser. Translates an expression like "A & B + !C" into a sorted list of region bitmasks. Grammar:

Operator	Meaning
~ or !	Complement (unary)
&	Intersection
or +	Union
(,)	Grouping

Atoms are uppercase letters A..I (one per set).

```
parse_region_expression("A & B", n_sets=3)
# -> [3, 7]
parse_region_expression("(A | B) & C", n_sets=3)
# -> [5, 6, 7]
parse_region_expression("A & ~B", n_sets=3)
# -> [1, 5]
```

CLI surface

```
# Items in every named set, written to stdout.
vdl data items dataset_real_cancer_drivers_4 \
    --mode intersection --sets Vogelstein,COSMIC_CGC,OncoKB --out -

# Items in this exact combination only.
vdl data items dataset_real_cancer_drivers_4 \
    --mode exclusive --sets A,B --out exclusive_AB.txt

# DSL debug / validator.
vdl data regions --expr "A & B + B & C" --n-sets 4
# -> 3,7,11,14,15

# Spotlight render with DSL highlight.
vdl render venn --sample --show-items \
    --highlight-expr "A & B & ~C & ~D" --out /tmp/spot.svg
```

Statistics (lower-level functions)

`compute_pairwise()` is what `analyze()` calls internally; the rest are scalar building blocks you can use independently in test code or custom pipelines.

```
from venn_diagram_lab.statistics import (
    jaccard, dice, overlap_coefficient,
    hypergeometric_p_value, fold_enrichment, bh_fdr,
    compute_pairwise,
)
```

Function	Returns	Notes
<code>jaccard(size_a, size_b, intersection)</code>	float in [0,1]	$ A \cap B / A \cup B $, 0 when both sets empty
<code>dice(size_a, size_b, intersection)</code>	float in [0,1]	$2 A \cap B / (A + B)$, the Sørensen–Dice coefficient
<code>overlap_coefficient(size_a, size_b, intersection)</code>	float in [0,1]	$ A \cap B / \min(A , B)$, Szymkiewicz–Simpson
<code>hypergeometric_p_value(K, n, k)</code>	float in [0,1]	One-sided over-representation $P(X \geq k)$
<code>fold_enrichment(N, K, n, k)</code>	float ≥ 0	$(k \cdot N) / (K \cdot n)$, observed / expected
<code>bh_fdr(p_values)</code>	list[float]	Benjamini–Hochberg adjusted p-values

Function	Returns	Notes
<code>compute_pairwise(set_names, inclusive_sizes, pairwise_intersections, universe_size)</code>	<code>StatisticsResult</code>	Roll-up of all of the above into matrices + a long table

```
# Hypergeometric: are 126 shared items unusual when set A has 138, B has 593, universe is 20000
hypergeometric_p_value(20000, 138, 593, 126)  # ~6.75e-184 (highly enriched)
fold_enrichment(20000, 138, 593, 126)         # ~30.8 fold over expected
```

Item share distribution & cluster ordering

```
from venn_diagram_lab.share_distribution import item_share_distribution
from venn_diagram_lab.cluster import cluster_set_order
```

```
item_share_distribution(matrix) -> dict[int, int]
```

Take a binary item×set numpy matrix; return {k: count} where k is the number of sets each item belongs to. Tall left bar (k=1) = set-specific signal dominates; tall right bar (k=N) = a core shared across every set.

```
import numpy as np
from venn_diagram_lab.share_distribution import item_share_distribution
matrix = np.array([[1,1,0],[1,1,1],[0,1,1],[1,0,0]])
item_share_distribution(matrix)           # {1: 1, 2: 2, 3: 1}
```

```
cluster_set_order(D, *, method="average") -> ClusterOrder
```

Hierarchical agglomerative clustering on a symmetric distance matrix ($D[i,j] \geq 0$, $D[i,i] = 0$). Methods: "average" (UPGMA, default), "complete", "single". Returns a `ClusterOrder` with `leaf_order` (left-to-right ordering of original indices) and `merges` (N-1 entries with `left`, `right`, `height`, `size`). The same ordering powers the cluster-heatmap renderer.

SVG renderers

All five SVG renderers return an `SvgImage(svg=str, width=int, height=int)`. Call `img.save(path)` to write to disk, or use `img.svg` directly as an in-memory string.

```
from venn_diagram_lab.render.svg import (
    render_venn_svg,
    render_share_distribution_svg,
    render_cluster_heatmap_svg,
    render_enrichment_bar_svg,
    render_enrichment_lollipop_svg,
)
```

+ also re-exported at the top level

```
render_venn_svg(result, *, model=None, set_names=None, colors=None,
title=None, show_names=True, show_counts=True, show_items=False,
item_options=None, highlight=None) -> SvgImage
```

Render the 44-model Venn template, substituting set names, counts, colours, and the optional title. `model=None` uses `result.model`. `set_names` overrides per-letter display names (`{"A": "Sample A"}`). `colors` overrides per-letter fills (`{"A": "#FFF200"}`). `show_names=False` blanks the labels; `show_counts=False` blanks the numbers.

```
img = render_venn_svg(result, title="Cancer drivers (4 sources)")
img.save("venn.svg")
```

Item display and spotlight parameters:

- `show_items=True` replaces each region's count with the actual item identifiers rendered as multi-line `<tspan>` content inside the existing `Count_*` text nodes.
- `item_options` is a mapping. Recognised keys (with defaults): `max_items_per_region=20`, `ncol_items=1`, `truncate_long_names=12` (0 disables), `line_height=10`, `font_size=8`, `show_counts_with_items=False`, `ellipsis="..."`. Unknown keys raise a `UserWarning`. Regions exceeding `max_items_per_region` show a trailing italic "+N more" line.
- `highlight` accepts either a sequence of region labels (e.g. `["AB", "ABC"]`) or a sequence of region bitmasks (e.g. `[3, 7]`, including the output of `parse_region_expression()`). Sets that do not contribute to any highlighted region are desaturated to `#cccccc` at 25% opacity.

```
img = render_venn_svg(res, show_items=True,
                      item_options={"max_items_per_region": 8,
                                   "truncate_long_names": 10})
masks = parse_region_expression("A & B & ~C & ~D", n_sets=4)
img = render_venn_svg(res, highlight=masks, show_items=True)
```

```
render_share_distribution_svg(dataset) -> SvgImage
```

A 480×280 viewBox histogram. Tier-gradient fill from `#ffe4b5` (`k=1`) to `#7e14ff` (`k=N`). Used as page 2 in the PDF report when `include_share=True` (default).

```
render_cluster_heatmap_svg(result, *, linkage="average", show_row_dendrogram=True,
show_col_dendrogram=True, dendrogram_fraction=0.12) -> SvgImage
```

Cluster-ordered Jaccard heatmap with L-shaped dendrograms. `dendrogram_fraction` controls how much of the canvas each dendrogram band gets (min 20 px effective).

```
render_enrichment_bar_svg(result, *, metric="neglog10fdr", width=560,
height=240) -> SvgImage
```

Pairwise enrichment bar chart. `metric` is "neglog10fdr" (default, $-\log_{10}(\text{BH-FDR})$) or "foldEnrichment". Green (#2e7d32) for $\text{FDR} < 0.05$, grey (#888888) otherwise. Significance markers (***) < 0.001 , ** < 0.01 , * < 0.05 above each bar.

```
render_enrichment_lollipop_svg(result, *, metric="neglog10fdr",
width=560, height=240) -> SvgImage
```

Same data + same colours as the bar chart, drawn as a stem-and-dot plot where the dot radius scales as $\sqrt{\text{intersection} / \text{max_intersection}}$ (range 2.5–8 px). Useful when you want to encode TWO numbers per pair (significance $\times$ overlap size) in one glyph.

```
generate_proportional_svg(result, *, width=600, height=600) ->
str
```

Area-proportional circles for 2- and 3-set inputs (analytic solve for $n=2$, binary search + Shapely union for $n=3$). Returns the raw SVG string. For 4+ sets the geometry problem is ill-posed; use a non-proportional model instead.

Matplotlib integration

Three SVG renderers ship parallel matplotlib variants returning an `MplImage(fig, legend)` wrapper. The SVG renderers stay unchanged so the web tool byte-equivalent contract is preserved; the matplotlib variants give you a `matplotlib.figure.Figure` you can drop into your own subplot grid or restyle with `plt.style.use(...)`.

```
render_venn_mpl(result, ax=None, model=None, set_names=None,
colors=None, title=None, show_names=True, show_counts=True,
alpha=0.45)
```

Matplotlib-native Venn for the 2-set, 3-set, and 4-set classic models plus the analytical `proportional` model (2 or 3 sets). Higher set counts (or template-based models like Edwards / Anderson / Mamakani) raise `IncompatibleModelError` with a pointer at `render_venn_svg`.

- `ax` — when supplied, draws into the given Axes; when `None`, creates a 7×7 figure internally.
- `model` — overrides the model. Allowed: "venn-2-set", "venn-3-set", "venn-4-set", "proportional". Default: canonical classic model for the dataset's set count.
- `colors` — per-letter (A..D) fill color override.
- `alpha` — fill alpha. Default 0.45.

```

import matplotlib.pyplot as plt
from venn_diagram_lab import render_venn_mpl, load_sample, analyze

res = analyze(load_sample("dataset_real_cancer_drivers_4"))

fig, ax = plt.subplots(1, 2, figsize=(12, 6))
render_venn_mpl(res, ax=ax[0], title="Default palette")
render_venn_mpl(res, ax=ax[1],
                 colors={"A": "#9b59b6", "B": "#3498db",
                        "C": "#e67e22", "D": "#1abc9c"},
                 title="Custom palette")
fig.tight_layout()

render_share_distribution_mpl(dataset, ax=None, color_low="#ffe4b5",
                             color_high="#7e14ff")

```

Matplotlib variant of the Item Share Distribution histogram. Same input contract as `render_share_distribution_svg`. Bars use a tier-gradient color scale matching the SVG variant. Returns an `MplImage` with an empty legend.

```

render_cluster_heatmap_mpl(result, ax=None, linkage="average",
                           show_row_dendrogram=True, show_col_dendrogram=True, cmap="Blues")

```

Matplotlib variant of the cluster-ordered Jaccard heatmap. Uses `scipy.cluster.hierarchy.dendrogram` for the side bands and `imshow` for the heatmap, plus a colorbar.

- When `ax` is `None`, returns a 7×6 figure with side dendrograms + a colorbar.
- When `ax` is supplied, draws only the heatmap into that Axes (single-panel mode; caller controls layout).
- `linkage` — "average" / "complete" / "single".

CLI `--backend` flag

`vd1 render venn`, `vd1 render share-dist`, and `vd1 render heatmap` each accept `--backend {svg,mpl}` (default `svg`). With `--backend mpl` the CLI dispatches to the matplotlib variant and writes a PNG (or PDF via `--out *.pdf`). The default `svg` behaviour is unchanged.

```

vd1 render venn --sample --backend mpl --out venn.png
vd1 render share-dist --sample --backend mpl --out share.png
vd1 render heatmap --sample --backend mpl --cluster --out heatmap.png

```

SvgImage Jupyter inline display

`SvgImage` (the return type of `render_venn_svg` and the other SVG renderers) implements both `_repr_svg_()` and `_repr_mimebundle_()`, so a notebook cell ending in an `SvgImage` value renders inline without `from IPython.display import SVG; display(SVG(img.svg))`.

ggplot/matplotlib renderers

These two renderers return `MplImage(fig=matplotlib.figure.Figure)`. Save the figure with `img.fig.savefig("out.png", dpi=200)` or pass to `matplotlib.pyplot.show()`.

```
render_upset(result, *, max_columns=20, sort_by="size", threshold=None,
color_mode="depth", colors=None) -> MplImage
```

UpSet plot: top bars = intersection sizes, left bars = set sizes, central matrix = membership dots. `sort_by` is "size" (default) or "degree". `color_mode` is "depth" (viridis on membership-degree), "heatmap" (Reds on size), or "custom" (use the `colors={label: hex}` map). `threshold` excludes intersections below the given size.

```
img = render_upset(result, max_columns=10, sort_by="degree", color_mode="heatmap")
img.fig.savefig("upset.png", dpi=200)
```

```
render_network(result, *, edge_metric="intersection", seed=42,
significance_threshold=0.05, node_color_map=None) -> MplImage
```

Force-directed network: nodes = sets (radius proportional to $\sqrt{\text{size}}$), edges = pairwise overlaps. `edge_metric` in {"intersection", "jaccard", "fold_enrichment", "overlap_coefficient"} controls edge thickness. Edges below `significance_threshold` (BH-FDR) are grey; significant edges are blue (#2E3192). `seed` makes the spring-embed layout reproducible. `node_color_map={"A": "#FFF200"}` overrides per-letter node fills.

Reports

```
render_pdf_report(result, path, *, title=None, include_network=True,
include_about=True, cluster_heatmap=False) -> None
```

The flagship multi-page PDF report. Pages produced (in order):

1. Overview (metadata + per-set size table)
2. Venn + UpSet side-by-side
3. Statistics (Jaccard, Dice, Enrichment tables — paginated when set count > 4)
4. Item Share Distribution (always on)
5. Cluster Heatmap (opt-in via `cluster_heatmap=True`)
6. Network + significant edges list (skip with `include_network=False`)
7. About / Credits / Cite (skip with `include_about=False`)

```
render_pdf_report(result, "report.pdf",
                  title="My analysis", cluster_heatmap=True)
```

```
to_excel_workbook(result, path) -> None
```

Three-sheet Excel workbook matching the *Download Everything* ZIP from the webtool:

- **Jaccard** — N×N matrix of Jaccard indices
- **Sørensen-Dice** — N×N matrix of Dice coefficients
- **Enrichment** — long-form `set_a`, `set_b`, `intersection`, `union`, `expected`, `fold_enrichment`, `p_value`, `fdr`, `significant`

```
from venn_diagram_lab.report.excel import to_excel_workbook
to_excel_workbook(result, "stats.xlsx")
```

Proportional layout

```
from venn_diagram_lab.proportional import (
    solve_2set, solve_3set, circle_intersection_area
)
```

```
solve_2set(a_only, b_only, ab) -> ProportionallLayout
```

Analytical 2-circle solver. `a_only` and `b_only` are the exclusive counts; `ab` is the intersection. Returns centre coordinates + radii so the rendered areas match the input ratios.

```
solve_3set(regions) -> ProportionallLayout
```

Three-circle solver. `regions` is the bitmask-keyed exclusive-count dictionary ({1: `a_only`, 2: `b_only`, 3: `ab`, 4: `c_only`, 5: `ac`, 6: `bc`, 7: `abc`}). Uses Shapely unions + binary search to converge on radii / centres.

```
circle_intersection_area(r1, r2, d) -> float
```

Lens area for two circles of radius `r1`, `r2` whose centres are at distance `d`. Returns 0 when disjoint, $\pi \cdot \min(r1, r2)^2$ when fully contained.

Data classes (@dataclass and TypedDict)

Class	Module	Key fields
Dataset	venn_diagram_lab.io	set_names: list[str], items: dict[str, list[str]], item_order: list[str], universe_size: int None, source_path: str None, format: str
RegionResult	venn_diagram_lab.analysis	dataset: Dataset, model: str, regions: dict[int, RegionData], set_sizes: dict[str, int], statistics: StatisticsResult
RegionData	venn_diagram_lab.analysis	mask: int, label: str, set_indices: tuple[int, ...], exclusive_items: list[str], inclusive_items: list[str]
StatisticsResult	venn_diagram_lab.statistics	jiaccard, dice, overlap_coefficient, fold_enrichment (all 4 are pd.DataFrame N×N), hypergeometric: pd.DataFrame long-form
ModelInfo	venn_diagram_lab.analysis	name: str, n_sets: int, family: str
SvgImage	venn_diagram_lab.rendering	svg str, width: int, height: int; .save(path)
MplImage	venn_diagram_lab.rendering	Figure matplotlib.figure.Figure; .save(path, dpi=200)

RegionResult also exposes export shortcuts:

```
result.to_region_summary_tsv("regions.tsv")
result.to_matrix_tsv("matrix.tsv")
```

```
result.to_statistics_tsv("statistics.tsv")
result.to_pdf_report("report.pdf")           # delegates to render_pdf_report
```

Exceptions

All raised from `venn_diagram_lab` (also importable from `venn_diagram_lab.errors`).

Exception	Raised when
<code>VennDiagramError</code>	Base class for all package errors
<code>InvalidDatasetError</code>	Bad input — wrong column count, mismatched header, parse error
<code>UnknownModelError</code>	<code>analyze(..., model="...")</code> or <code>render_venn_svg(model="...")</code> with an unknown stem
<code>IncompatibleModelError</code>	Requested model's <code>n_sets</code> doesn't match the dataset

CLI reference (vdl)

The CLI mirrors the library 1-for-1. Every subcommand accepts a dataset *path* or a bundled sample *name*; with `--sample` (no value), it picks the bundled cancer-drivers demo. Output goes to `--out PATH`; `--out -` writes to stdout.

The two top-level navigation commands are useful from any shell:

```
vdl --help           # short summary of every command
vdl tree             # tree view of every command + short-help
vdl about            # abbreviated "About Venn Diagrams"
vdl credits          # authors + citation + links
vdl version          # 2.4.1
```

Top-level shortcuts

Command	Equivalent	Purpose
<code>vdl analyze INPUT</code>	<code>report pdf + export</code>	One-shot: analyze +
<code>[--out DIR]</code>	<code>*</code>	write all artefacts
<code>vdl render-sample</code>	<code>analyze --sample</code>	Same as analyze but
<code>NAME [--out DIR]</code>	<code>SAMPLE</code>	takes a sample name
<code>vdl list-models</code>	<code>model list</code>	One name per line
<code>vdl list-samples</code>	<code>data samples</code>	One name per line
<code>vdl cluster INPUT</code>	<code>render heatmap</code>	Convenience for
<code>--out heatmap.svg</code>	<code>--cluster</code>	cluster-heatmap
<code>vdl share-dist INPUT</code>	<code>render share-dist</code>	Convenience for
<code>--out hist.svg</code>		share-distribution

vdl data

Command	Synopsis	Example
<code>vdl data samples</code>	List bundled sample names, one per line	<code>vdl data samples</code>
<code>vdl data validate</code> <code>INPUT [--strict]</code> <code>[--json]</code>	Schema + content validation; <code>--json</code> for machine-readable output	<code>vdl data validate genes.tsv --json</code>
<code>vdl data describe</code> <code>INPUT</code>	Quick text summary (set count, item counts, universe size, source path)	<code>vdl data describe genes.tsv</code>
<code>vdl data convert</code> IN OUT	TSV <-> CSV conversion (delimiter detected by extension)	<code>vdl data convert genes.tsv genes.csv</code>
<code>vdl data fit-model</code> <code>INPUT</code>	Recommend a model name for the dataset's set count	<code>vdl data fit-model genes.tsv</code>
<code>vdl data lookup</code> <code>INPUT ITEM</code>	Which Venn region(s) does ITEM belong to?	<code>vdl data lookup genes.tsv TP53</code>
<code>vdl data items</code> INPUT <code>--mode</code> <code>{intersection,exclusive,union}</code> <code>--sets A,B [--out -]</code>	List items matching a set-combination query	<code>vdl data items genes.tsv --mode exclusive --sets A,B --out -</code>
<code>vdl data regions</code> <code>--expr "A & B"</code> <code>--n-sets 4 [--format</code> <code>masks\ labels]</code>	Resolve a Boolean DSL expression to a mask vector	<code>vdl data regions --expr "A & B" --n-sets 4</code>

vdl export

All four commands take `--out PATH` (or `--out -` for stdout) and `--model MODEL` (or `--model auto`).

Command	Output
<code>vdl export region-summary</code> INPUT <code>--out regions.tsv</code>	Per-region exclusive + inclusive counts
<code>vdl export matrix</code> INPUT <code>--out</code> <code>matrix.tsv</code>	Binary item $\times$ set membership table

Command	Output
<code>vdl export statistics INPUT --out stats.tsv</code>	Long-form pairwise table: set_a, set_b, intersection, expected, p_value, fdr, jaccard, dice, overlap_coefficient, fold_enrichment, significant
<code>vdl export pairwise INPUT --out stats.tsv</code>	Alias of <code>statistics</code> (bioinformatics naming)

vdl model

Command	Purpose
<code>vdl model list</code>	Bundled model names, one per line (use to discover what's available)
<code>vdl model info NAME</code>	Print n_sets, family, file path
<code>vdl model svg NAME --out template.svg</code>	Write the raw SVG template (no result substitution)

vdl render

All renderers accept `INPUT` (or `--sample`), `--model MODEL`, and `--out PATH` (default is a name derived from the input). Format is inferred from extension (`.svg`, `.png`, `.pdf`).

Command	Flags	Notes
<code>vdl render venn INPUT --out venn.svg</code>	<code>--title TXT</code> , <code>--no-names</code> , <code>--no-counts</code> , <code>--colors</code> <code>'{...}'</code> , <code>--show-items</code> , <code>--max-items-per-region</code> <code>N</code> , <code>--truncate-long-names</code> <code>N</code> , <code>--highlight</code> <code>"AB,ABC"</code> , <code>--highlight-expr "A</code> <code>& B"</code>	The Venn template; <code>--show-items</code> writes item identifiers into each region; <code>--highlight /</code> <code>--highlight-expr</code> spotlight the named or DSL-resolved regions
<code>vdl render upset INPUT --out upset.png</code>	<code>--max-columns N</code> , <code>--sort-by</code> <code>{size,degree}</code> , <code>--threshold N</code> , <code>--color-mode</code> <code>{depth,heatmap,custom}</code>	UpSet plot

Command	Flags	Notes
<code>vdl render network</code> <code>INPUT --out</code> <code>network.png</code>	<code>--edge-metric</code> <code>{intersection,jaccard,fold_enrichment,overlap_coefficient},</code> <code>--significance-threshold</code> <code>F, --seed N</code>	Force-directed network
<code>vdl render heatmap</code> <code>INPUT --out hm.svg</code>	<code>--cluster /</code> <code>--no-cluster,</code> <code>--linkage</code> <code>{average,complete,single}</code> (no extras)	Heatmap; with <code>--cluster</code> it's the dendrogrammed one
<code>vdl render</code> <code>share-dist INPUT</code> <code>--out hist.svg</code>		Item share distribution histogram
<code>vdl render bar INPUT</code> <code>--out bar.svg</code>	<code>--metric</code> <code>{neglog10fdr,foldEnrichment},</code> <code>--width N, --height N</code>	Enrichment bar chart
<code>vdl render lollipop</code> <code>INPUT --out lol.svg</code>	<code>--metric</code> <code>{neglog10fdr,foldEnrichment},</code> <code>--width N, --height N</code> (no extras)	Enrichment lollipop
<code>vdl render all INPUT</code> <code>--out-dir DIR</code>		Convenience: Venn + UpSet + Network + Heatmap + Share-Dist into one directory

vdl report

Command	Output	Flags
<code>vdl report pdf INPUT</code> <code>--out report.pdf</code>	Multi-page PDF (overview, venn+upset, statistics, share-dist, network, about)	<code>--no-network,</code> <code>--no-about,</code> <code>--cluster-heatmap</code>
<code>vdl report zip INPUT</code> <code>--out bundle.zip</code>	ZIP with PDF + 4 SVGs + 3 TSVs + xlsx + README.txt	(same flags as <code>pdf</code> pass through to the inner PDF)

vdl workflow

Command	Purpose
<code>vdl workflow init DIR</code>	Scaffold <code>DIR/data/</code> , <code>DIR/output/</code> , <code>DIR/analysis.yaml</code> with a working config

Command	Purpose
<code>vd1 workflow bench INPUT</code> <code>[--repeats N]</code>	Per-stage timing: load -> analyze -> render_ <i>/export_</i> -> report; useful for profiling pipelines
<code>vd1 workflow run-from analysis.yaml</code>	Execute every step described in the YAML (replicates a full pipeline)

YAML config schema (workflow run-from)

```

version: 1
dataset: data/genes.tsv           # or "@sample:dataset_real_cancer_drivers_4"
model: auto                       # or e.g. venn-4-set
output: output/
report:
  pdf: report.pdf                 # all report keys are optional
  zip: bundle.zip
  cluster_heatmap: true
exports:
  region-summary: regions.tsv
  matrix: matrix.tsv
  statistics: stats.tsv
renders:
  venn: venn.svg
  upset: upset.png
  network: network.png
  heatmap: heatmap.svg
  share-dist: hist.svg
  bar: bar.svg
  lollipop: lol.svg

```

Run with:

```
vd1 workflow run-from analysis.yaml
```

Common workflows

End-to-end PDF report from a TSV

```
vd1 report pdf data/genes.tsv --out output/report.pdf --cluster-heatmap
```

...or from Python:

```

from venn_diagram_lab import load_tsv, analyze, render_pdf_report
result = analyze(load_tsv("data/genes.tsv"))
render_pdf_report(result, "output/report.pdf", cluster_heatmap=True)

```

Full bundle (PDF + SVGs + TSVs + xlsx + README)
for a downstream user

```
vd1 report zip data/genes.tsv --out bundle.zip
```

CI parity test: compare two TSV exports byte-by-byte

```
vd1 export statistics data/genes.tsv --out new.tsv
diff -q baseline/stats.tsv new.tsv && echo "byte-identical"
```

Significance enrichment in a notebook

```
from venn_diagram_lab import load_sample, analyze
result = analyze(load_sample("dataset_real_cancer_drivers_4"))
sig = result.statistics.hypergeometric.query("significant == True").sort_values("p_adjusted")
sig.head()
```

#	set_a	set_b	intersection	expected	p_value	p_adjusted	significant
#	COSMIC_CGC	OncoKB	581	35.76	0.0	0.0	True
#	COSMIC_CGC	IntOGen	388	18.39	0.0	0.0	True
#	OncoKB	IntOGen	477	38.96	0.0	0.0	True

Custom 4-letter colour mapping in the Venn

```
from venn_diagram_lab import analyze, load_sample, render_venn_svg
result = analyze(load_sample("dataset_real_cancer_drivers_4"))
custom = {"A": "#FF6B6B", "B": "#4ECDC4", "C": "#FFE66D", "D": "#1A535C"}
img = render_venn_svg(result, colors=custom, title="My palette")
img.save("custom.svg")
```

Output formats

Format	Surface	Spec
TSV (region-summary / matrix / statistics)	<code>result.to_*_tsv()</code> , <code>vd1 export *</code>	UTF-8, LF line endings, = / + / - / @ prefixes escaped to prevent CSV-injection in spreadsheets
SVG (Venn / share-dist / heatmap / bar / lollipop)	<code>render_*_svg()</code> , <code>vd1 render *</code>	Plain SVG 1.1 viewBox, no external CSS or scripts, byte-equivalent to the webtool's output
PNG (UpSet / Network)	<code>MplImage.fig.savefig()</code> , <code>vd1 render upset/network with --out *.png</code>	Matplotlib raster at requested DPI (default 200)

Format	Surface	Spec
PDF (report)	<code>render_pdf_report()</code> , <code>vdl report pdf</code>	US Letter landscape, multi-page via matplotlib <code>PdfPages</code>
Excel (statistics workbook)	<code>to_excel_workbook()</code> , embedded in ZIP	3 sheets: Jaccard, Sørensen-Dice, Enrichment; bold/grey header rows + 4-decimal numeric format
ZIP (Download Everything)	<code>vdl report zip</code>	PDF + 4 SVGs + 3 TSVs + xlsx + README.txt, flat layout

Cross-package parity

This Python package, the R companion `vennDiagramLab`, and the webtool all ship in lock-step. TSV exports are tested byte-equivalent across all three; PDF/SVG/PNG are functionally equivalent but pixel-level differences exist due to different rendering backends (matplotlib vs grDevices vs browser SVG).

Functionality	Python	R	Webtool
5 loaders (CSV/TSV/GMT/GMX/sample)	Yes	Yes	CSV/TSV/GMT/GMX via dialog
44 SVG models	Yes	Yes	Yes + interactive editor
analyze + all statistics	Yes	Yes	Yes
7 SVG renderers	Yes	Yes	Yes
UpSet + Network (matplotlib / ggplot2 / browser)	Yes	Yes	Yes
4-page PDF report core	Yes	Yes	Yes
Item Share	Yes	Yes	Yes
Distribution page			
Cluster Heatmap page	Yes (opt-in)	Yes (opt-in)	Yes (toggle)
ZIP bundle	Yes	Yes	Yes (Download Everything)
Excel workbook	Yes	Yes	Yes

Functionality	Python	R	Webtool
CLI (<code>vd1 ...</code>)	Yes	(R has no idiomatic CLI; the lib is the surface)	N/A
<code>ggplot2</code>	—	Yes	N/A
<code>geom_venn()</code>	—	Yes	N/A
<code>broom</code>	—	Yes	N/A
<code>tidy()/glance()/augment()</code>	—	—	—
Workflow YAML runner	Yes	(use <code>Rscript</code> directly)	N/A
In-region item display	Yes	Yes	No
(<code>show_items</code>)	—	—	—
Spotlight highlight	Yes	Yes	No
(<code>highlight=[...]</code>)	—	—	—
Region accessors	Yes	Yes	No
(<code>intersection_items</code> / <code>exclusive_items</code> / <code>union_items</code>)	—	—	—
Boolean DSL	Yes	Yes	No
(<code>parse_region_expression</code>)	—	—	—

Troubleshooting

Symptom	Cause	Remedy
<code>InvalidDatasetError: header has 5 columns, expected >= 2</code>	First line of CSV/TSV not a valid set-name header	Check that row 1 contains the set names
<code>UnknownModelError: 'venn-foo' not found</code>	Misspelled model name	<code>vd1 model list</code> to discover the 44 valid names
<code>IncompatibleModelError: model has 5 sets, dataset has 4</code>	Explicit <code>--model</code> mismatches the dataset's set count	Use <code>--model auto</code> or pick a model with matching <code>n_sets</code>
PDF report missing a page	<code>--no-network</code> / <code>--no-about</code> flag set, or <code>cluster_heatmap=False</code> (default)	Re-run without the flag, or pass <code>--cluster-heatmap</code>

Symptom	Cause	Remedy
<code>ModuleNotFoundError: 'cairosvg' on import</code>	cairosvg system dep (Cairo) not installed	Install Cairo system library, then <code>pip install --force-reinstall cairosvg</code>
<code>tableGrob</code> warnings during <code>render_pdf_report</code>	Set names contain non-ASCII / very long strings	Cosmetic only — the PDF still renders; truncate set names to ≤ 30 chars to silence
<code>ValueError: Unknown set identifier: 'Z' from a region accessor</code>	Set letter or display name does not match <code>result.dataset.set_names</code>	List <code>result.dataset.set_names</code> to see valid identifiers; letters are A..I in dataset order
<code>ValueError: highlight: no region with mask 99 from render_venn_svg</code>	A bitmask outside <code>1..(2^n_sets - 1)</code> was passed to <code>highlight</code>	Use <code>parse_region_expression()</code> or check <code>1..(2^n - 1)</code> bounds
<code>ValueError: Malformed region expression: trailing operator</code>	A Boolean DSL string ends with a dangling <code>&</code> / <code>+</code> / <code>~</code>	Complete the expression — every operator needs operands on both sides
<code>ValueError: Parenthesis mismatch in region expression</code>	Unbalanced (or) in the Boolean DSL string	Recount opening vs. closing parentheses
<code>UserWarning: Ignoring unknown item_options: 'bogus'</code>	A typo in <code>item_options</code> keys	Recognised keys: <code>max_items_per_region</code> , <code>ncol_items</code> , <code>truncate_long_names</code> , <code>line_height</code> , <code>font_size</code> , <code>show_counts_with_items</code> , <code>ellipsis</code>

References

- **Web tool:** <https://venndiagramlab.org/>
- **Python package:** <https://pypi.org/project/venn-diagram-lab/>
- **R companion (CRAN):** <https://CRAN.R-project.org/package=vennDiagramLab>
- **GitHub monorepo:** <https://github.com/ZoliQua/Venn-Diagram-Lab>
- **Zenodo concept DOI:** <https://doi.org/10.5281/zenodo.19510813>

Citation

Dul Z., Ölbei M., Thomas N.S.B., Si Ammour A., Csikász-Nagy A. (2026). *Venn Diagram Lab — Headless Venn diagram analysis and rendering*. Python package version 2.4.1. <https://venndiagramlab.org/> doi:10.5281/zenodo.19510813